გაკვეთილი 6 Mod 4 (4.2)

გაკვეთილი 4 — TypeScript-ში ოპერატორები და გამოსახულებები

1. წინა გაკვეთილის გახსენება

წინა გაკვეთილში ვისწავლეთ მონაცემთა ტიპების კონვერტაცია.

მაგალითად:

```
let firstNumber: string = "20";
let secondNumber: string = "10";

let a: number = Number(firstNumber);
let b: number = Number(secondNumber);

console.log(a + b);
```

შედეგი:

```
30
```

აქ ტექსტური მნიშვნელობები რიცხვებად გარდავქმენით და შემდეგ შევკრიბეთ.

დღეს ვნახავთ, როგორ მუშაობენ სხვადასხვა ოპერატორები TypeScript-ის ტიპებთან.

2. არითმეტიკული ოპერატორები — Arithmetic Operators

JavaScript-იდან უკვე იციან ძირითადი არითმეტიკული ოპერატორები.

ოპერატორი	დანიშნულება	მაგალითი
`+`	შეკრება	`10 + 5`
`-`	გამოკლება	`10 - 5`
`*`	გამრავლება	`10 * 5`
`/`	გაყოფა	`10 / 5`
`%`	ნაშთის პოვნა	`10 % 3`
`**`	ხარისხში აყვანა	`2 ** 3`

მაგალითი 1 — რიცხვებზე მოქმედებები

```
let a: number = 20;
let b: number = 5;

let sum: number = a + b;
let difference: number = a - b;
let product: number = a * b;
let division: number = a / b;
let remainder: number = a % b;

console.log(sum);
console.log(difference);
console.log(product);
console.log(division);
console.log(remainder);
```

შედეგი:

```
25
15
100
4
0
```

ამ კოდში ყველა ცვლადს მითითებული აქვს `number` ტიპი.

TypeScript წინასწარ ამოწმებს, რამდენად შეესაბამება მოქმედებაში გამოყენებული მნიშვნელობები ოპერატორის მოთხოვნებს.

მაგალითი 2 — TypeScript-ის შეცდომა

```
let a: number = 20;
let b: string = "5";

let result: number = a - b;
```

TypeScript შეცდომას გვაჩვენებს.

მიზეზი: `b` არის `string`, ხოლო გამოკლების ოპერატორი TypeScript-ში რიცხვით მონაცემებს მოითხოვს.

JavaScript ამ შემთხვევაში შეცდებოდა `"5"`-ის ავტომატურად გარდაქმნას რიცხვად, მაგრამ TypeScript ასეთ კოდს ტიპების შემოწმებისას არ უშვებს.

სწორი ვარიანტი:

```
let a: number = 20;
let b: string = "5";

let result: number = a - Number(b);

console.log(result);
```

შედეგი:

```
15
```

მაგალითი 3 — `+` ოპერატორის თავისებურება

`+` ოპერატორი გამოიყენება როგორც რიცხვების შესაკრებად, ასევე ტექსტების გასაერთიანებლად.

```
let a: number = 10;
let b: string = "20";

let result: string = a + b;

console.log(result);
```

შედეგი:

```
1020
```

რატომ?

იმიტომ, რომ ერთ-ერთი მნიშვნელობა `string` ტიპისაა და `+` ტექსტების გაერთიანებას ასრულებს.

თუ გვინდა მათემატიკური შეკრება:

```
let result: number = a + Number(b);

console.log(result);
```

შედეგი:

```
30
```

მნიშვნელოვანი: TypeScript-ში `+`-ის გამოყენება `number`-სა და `string`-ს შორის დაშვებულია, მაგრამ შედეგი ტექსტია. ამიტომ ყოველთვის უნდა დავაკვირდეთ, რა ტიპის შედეგს ვიღებთ.

3. მინიჭების ოპერატორები — Assignment Operators

მინიჭების ოპერატორები ცვლადის მნიშვნელობის შეცვლისთვის გამოიყენება.

ოპერატორი	მაგალითი	იგივეა, რაც
`=`	`a = 10`	მნიშვნელობის მინიჭება
`+=`	`a += 5`	`a = a + 5`
`-=`	`a -= 5`	`a = a - 5`
`*=`	`a *= 5`	`a = a * 5`
`/=`	`a /= 5`	`a = a / 5`
`%=`	`a %= 5`	`a = a % 5`

მაგალითი 1 — ქულების დამატება

```
let score: number = 100;

score += 20;

console.log(score);
```

შედეგი:

```
120
```

მაგალითი 2 — თამაშის სიცოცხლეები

```
let lives: number = 5;

lives -= 1;

console.log(lives);
```

შედეგი:

```
4
```

TypeScript-ის თავისებურება

```
let score: number = 100;

score += "20";
```

ეს შეცდომას გამოიწვევს.

რატომ?

რადგან `score` არის `number`, ხოლო `"20"` არის `string`. `+=` ამ შემთხვევაში ტექსტურ გაერთიანებას გამოიწვევდა, რის შედეგადაც მიღებული მნიშვნელობა აღარ იქნებოდა `number`.

სწორი ვარიანტი:

```
let score: number = 100;

score += Number("20");

console.log(score);
```

შედეგი:

```
120
```

4. ინკრემენტი და დეკრემენტი

ეს ოპერატორები JavaScript-იდან უკვე ნაცნობია.

`++` ზრდის მნიშვნელობას ერთით.

`--` ამცირებს მნიშვნელობას ერთით.

```
let count: number = 5;

count++;

console.log(count);
```

შედეგი:

```
6
```

მეორე მაგალითი:

```
let lives: number = 3;

lives--;

console.log(lives);
```

შედეგი:

```
2
```

TypeScript-ში მნიშვნელოვანია, რომ ასეთ ოპერატორებს შესაბამისი რიცხვითი ტიპის მნიშვნელობებზე ვიყენებდეთ.

მაგალითად:

```
let count: string = "5";

count++;
```

ეს TypeScript-ში შეცდომაა.

სწორი ვარიანტი:

```
let count: number = Number("5");

count++;
```

5. შედარების ოპერატორები — Comparison Operators

შედარების ოპერატორები გამოიყენება ორი მნიშვნელობის შესადარებლად.

შედარების შედეგი ხშირად არის `boolean` ტიპის მნიშვნელობა:

```
true
false
```

ოპერატორი	მნიშვნელობა
`>`	მეტია
`<`	ნაკლებია
`>=`	მეტია ან ტოლია
`<=`	ნაკლებია ან ტოლია
`==`	მნიშვნელობების არამკაცრი შედარება
`===`	მკაცრი ტოლობა
`!=`	არამკაცრი არატოლობა
`!==`	მკაცრი არატოლობა

მაგალითი 1 — რიცხვების შედარება

```
let a: number = 20;
let b: number = 10;

let result: boolean = a > b;

console.log(result);
```

შედეგი:

```
true
```

აქ `result` არის `boolean`, რადგან შედარების შედეგად ვიღებთ `true` ან `false` მნიშვნელობას.

მაგალითი 2 — ასაკის შემოწმება

```
let age: number = 17;

let isAdult: boolean = age >= 18;

console.log(isAdult);
```

შედეგი:

```
false
```

აქ `if` ოპერატორი არ გვჭირდება, რადგან შედარება პირდაპირ გვიბრუნებს `boolean` მნიშვნელობას.

6. `==` და `===` — მნიშვნელოვანი განსხვავება

ეს JavaScript-იდან ნაცნობი თემაა, მაგრამ TypeScript-ში განსაკუთრებული ყურადღება უნდა მივაქციოთ.

JavaScript-ში:

```
console.log(10 == "10");
```

შედეგია:

```
true
```

რადგან `==` შედარებისას შეიძლება ტიპების ავტომატური გარდაქმნა მოხდეს.

ხოლო:

```
console.log(10 === "10");
```

შედეგია:

```
false
```

რადგან `===` მნიშვნელობასთან ერთად ტიპსაც ითვალისწინებს.

რა ხდება TypeScript-ში?

```
let a: number = 10;
let b: string = "10";

console.log(a === b);
```

TypeScript შეცდომას გვაჩვენებს, რადგან `number` და `string` ერთმანეთისგან განსხვავებული ტიპებია და მათ შესაძლო მნიშვნელობებს გადაკვეთა არ აქვთ.

თუ გვინდა მათი რიცხვებად შედარება:

```
let a: number = 10;
let b: string = "10";
```

```
console.log(a === Number(b));
```

შედეგი:

```
true
```

რეკომენდაცია: TypeScript-ში უპირატესობა მიანიჭეთ `===` და `!==` ოპერატორებს. საჭიროების შემთხვევაში ტიპები წინასწარ გარდაქმენით.

7. ლოგიკური ოპერატორები — Logical Operators

ლოგიკური ოპერატორები გამოიყენება რამდენიმე პირობის გასაერთიანებლად.

ძირითადი ოპერატორებია:

ოპერატორი	სახელი	დანიშნულება
`&&`	AND	ორივე პირობა უნდა შესრულდეს
`\`	\	`
`!`	NOT	აბრუნებს ლოგიკურ მნიშვნელობას

`&&` — AND

```
let age: number = 20;
let hasTicket: boolean = true;

let canEnter: boolean = age >= 18 && hasTicket;

console.log(canEnter);
```

შედეგი:

```
true
```

რადგან ორივე პირობა შესრულებულია.

AND — ორივე პირობა უნდა იყოს true

A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

`||` — OR

```
let isTeacher: boolean = false;
let isAdmin: boolean = true;

let hasAccess: boolean = isTeacher || isAdmin;

console.log(hasAccess);
```

შედეგი:

```
true
```

რადგან ერთი პირობა მაინც შესრულებულია.

A	B	A | B
true	true	true
true	false	true
false	true	true
false	false	false

`!` — NOT

`!` ოპერატორი ლოგიკურ მნიშვნელობას საპირისპიროთი ცვლის.

```
let isOnline: boolean = true;

let isOffline: boolean = !isOnline;

console.log(isOffline);
```

შედეგი:

```
false
```

კიდევ ერთი მაგალითი:

```
let gameOver: boolean = false;

console.log(!gameOver);
```

შედეგი:

```
true
```

8. TypeScript-ში ლოგიკური ოპერატორების თავისებურება

JavaScript-ში `&&` და `||` ოპერატორები ყოველთვის `boolean` მნიშვნელობას არ აბრუნებენ.

მაგალითად:

```
let username: string = "";

let result = username || "Guest";

console.log(result);
```

შედეგი:

```
Guest
```

აქ `||` აბრუნებს პირველ truthy მნიშვნელობას, ხოლო თუ ასეთი არ არსებობს — ბოლო მნიშვნელობას.

TypeScript ამ შემთხვევაშიც განსაზღვრავს შედეგის ტიპს.

მაგალითად:

```
let username: string = "";

let displayName: string = username || "Guest";
```

ეს სწორია.

მაგრამ:

```
let username: string = "";

let displayName: number = username || "Guest";
```

არასწორია, რადგან მიღებული შედეგი `string` ტიპისაა.

`??` — Nullish Coalescing

TypeScript-ში ხშირად გამოიყენება კიდევ ერთი ოპერატორი:

```
??
```

ის JavaScript-შიც არსებობს და მნიშვნელობის ჩანაცვლებისთვის გამოიყენება მაშინ, როდესაც მნიშვნელობა არის `null` ან `undefined`.

მაგალითად:

```
let username: string | null = null;

let displayName: string = username ?? "Guest";

console.log(displayName);
```

შედეგი:

```
Guest
```

შევადაროთ `||`-ს:

```
let username: string = "";

console.log(username || "Guest");
console.log(username ?? "Guest");
```

შედეგი:

```
Guest
```

მეორე `console.log()` ცარიელ ტექსტს გამოიტანს.

განსხვავება: `||` ცარიელ ტექსტსაც ჩაანაცვლებს, ხოლო `??` მხოლოდ `null`-სა და `undefined`-ს.

9. TypeScript და Union Type ოპერატორებთან

წინა გაკვეთილში ვისწავლეთ Union Type.

```
let value: number | string = 20;
```

ზოგჯერ TypeScript-ს სჭირდება დარწმუნდეს, კონკრეტულად რომელ ტიპთან მუშაობს.

მაგალითად:

```
function doubleValue(value: number | string) {
    return value * 2;
}
```

ეს შეცდომაა, რადგან `value` შეიძლება იყოს `string`.

შეგვიძლია გამოვიყენოთ `typeof`:

```
function doubleValue(value: number | string): number {
    if (typeof value === "string") {
        return Number(value) * 2;
    }

    return value * 2;
}

console.log(doubleValue(10));
console.log(doubleValue("15"));
```

შედეგი:

```
20
30
```

აქ TypeScript `typeof` შემოწმების საფუძველზე განსაზღვრავს, რომელ ტიპთან გვაქვს საქმე.

ამ პროცესს Type Narrowing ეწოდება.

ფუნქციები და `if` JavaScript-იდან უკვე იციან, ამიტომ აქ ყურადღება TypeScript-ის ტიპების შემოწმებაზე გაამახვილეთ.

10. პრაქტიკული სავარჯიშოები

სავარჯიშო 1 — არითმეტიკული ოპერატორები

მოცემულია:

```
let a: string = "30";
let b: string = "10";
```

დავალება: შექმენით ოთხი `number` ტიპის ცვლადი, რომლებშიც შეინახავთ ამ მნიშვნელობების ჯამს, სხვაობას, ნამრავლსა და განაყოფს.

შედეგები გამოიტანეთ კონსოლში.

სავარჯიშო 2 — იპოვე შეცდომები

მოცემულია TypeScript კოდი:

```
let price: number = 100;
let discount: string = "20";

let finalPrice: number = price - discount;

console.log(finalPrice);
```

დავალება: იპოვეთ შეცდომა და გაასწორეთ ისე, რომ საბოლოო ფასი იყოს `80`.

სწორი პასუხის ნახვა

სავარჯიშო 3 — შედარების ოპერატორები

შექმენით შემდეგი ცვლადები:

```
let age: number = 17;
let score: number = 85;
```

დაწერეთ კოდი, რომელიც შეამოწმებს:

1. არის თუ არა ასაკი 18-ზე მეტი ან ტოლი;
2. არის თუ არა ქულა 50-ზე მეტი;

3. არის თუ არა ქულა ზუსტად 100-ის ტოლი.

სამივე შედეგი შეინახეთ `boolean` ტიპის ცვლადებში და გამოიტანეთ კონსოლში.

სავარჯიშო 4 — ლოგიკური ოპერატორები

მოცემულია:

```
let age: number = 20;
let hasTicket: boolean = true;
let isBanned: boolean = false;
```

დავალება: შექმენით `canEnter` ცვლადი, რომელიც იქნება `true` მხოლოდ მაშინ, თუ:

- მომხმარებელი არის მინიმუმ 18 წლის;
- აქვს ბილეთი;
- არ არის დაბლოკილი.

გამოიყენეთ `&&` და `!`.

სწორი პასუხის ნახვა

სავარჯიშო 5 — Union Type და ოპერატორები

მოცემულია:

```
let firstValue: number | string = "50";
let secondValue: number = 20;
```

დავალება: გარდაქმენით პირველი მნიშვნელობა რიცხვად და გამოთვალეთ მათი ჯამი.

შედეგი უნდა იყოს `70`.

11. დამატებითი პრაქტიკული დავალება — ონლაინ მაღაზია

შექმენით TypeScript პროგრამა ონლაინ მაღაზიისთვის.

პროგრამაში უნდა არსებობდეს:

- `productName` — პროდუქტის სახელი;
- `price` — პროდუქტის ფასი;
- `quantity` — რაოდენობა;
- `discount` — ფასდაკლების თანხა;

- `isAvailable` — არის თუ არა პროდუქტი მარაგში.

პროგრამამ უნდა:

1. გამოთვალოს ჯამური ფასი.
2. გამოაკლოს ფასდაკლება.
3. შეამოწმოს, არის თუ არა საბოლოო ფასი 100-ზე მეტი.
4. შეამოწმოს, შესაძლებელია თუ არა შეძენა — პროდუქტი უნდა იყოს მარაგში და რაოდენობა 0-ზე მეტი.
5. ყველა შედეგი გამოიტანოს კონსოლში.